

06 — OOP, VIRTUAL DISPATCH & POLYMORPHISM INTERNALS

Beyond "what is inheritance" — interviewers want the mechanics (vtables), the pitfalls (slicing, virtual dtors), and when *not* to use inheritance.

How to think about this. A virtual call answers "which function?" by looking at the object's *runtime* type instead of the pointer's *compile-time* type. Physically that's one hidden pointer per object (the vptr) and one table per class (the vtable). Hold that single picture in your head and every pitfall becomes obvious: slicing = copied into a base, lost the vptr; virtual-dtor rule = the vtable must route `delete` to the real destructor; ctor/dtor dispatch = the vptr isn't set to the derived table yet. Draw the boxes and arrows once.

Take-home lesson. `virtual` buys you one thing — a runtime choice of function based on the object's *dynamic* type — and charges you a vptr per object plus an indirection per call. Everything else about C++ polymorphism (slicing, the virtual-dtor rule, static-vs-dynamic dispatch) falls out of understanding that single mechanism.

1. VIRTUAL DISPATCH MECHANICS (VTABLE / VPTR)

- A class with any `virtual` function gets a hidden vptr (usually first member) pointing at the class's vtable — an array of function pointers, one per virtual function.
- A virtual call = deref vptr → index into vtable → indirect call. One extra indirection; usually can't be inlined (unless the dynamic type is known / devirtualized).

- vtable is per-class, created once; vptr is per-object, set by the constructor. Cost: one pointer per object + one indirection per call.
- `sizeof` a polymorphic class includes the vptr.

Calling virtuals in ctor/dtor: dispatch uses the *currently constructed* type, not the most-derived. During `Base`'s ctor the object *is* a `Base` — a virtual call resolves to `Base`'s version. Never rely on derived overrides from within a base ctor/dtor.

2. THE VIRTUAL DESTRUCTOR RULE (ALWAYS ASKED)

If a class is meant to be deleted through a base pointer, its destructor must be virtual — otherwise `delete basePtr` on a derived object is UB (only `~Base` runs; derived resources leak).

```
struct Base { virtual ~Base() = default; }; // enables correct
polymorphic delete
```

- Corollary: a class with virtuals but a non-virtual dtor is a design red flag.
- If a class isn't meant to be a base, don't pay for a vptr — or mark it `final`.

Stop and think. `std::unique_ptr<Base> p = std::make_unique<Derived>();`

When `p` goes out of scope, does `~Derived` run if `~Base` is non-virtual?

► answer

3. OVERRIDE CORRECTNESS

- `override` — compiler-checked: errors if it doesn't actually override (wrong signature, missing `const`, etc.). Always use it.
- `final` — forbids further overriding (or subclassing). Can enable

devirtualization.

- Overriding hides all base overloads of the same name unless you `using Base::f;`. Name hiding surprises people.
 - Covariant return types are allowed (override may return a more-derived ptr/ref).
 - Default arguments are bound statically (by the pointer's static type), while the body dispatches dynamically — mixing them is a trap. Don't give virtuals different defaults across overrides.
-

4. SLICING

Copying a derived object into a base *value* drops the derived part (and the vptr becomes Base's). Passing polymorphic objects by value slices them.

```
void take(Base b);           // slices any Derived passed in
void take(const Base& b);    // correct — preserves dynamic type
```

5. INHERITANCE FLAVORS & ALTERNATIVES

- public = "is-a" (substitutability, LSP). private/protected = "implemented in terms of" — prefer *composition* for that instead.
- Abstract classes / pure virtual (`= 0`) define interfaces. A pure virtual can still have a body (a default impl callable via `Base::f()`).
- Multiple inheritance → the diamond problem. Solve with virtual inheritance (`class D : virtual Base`) so there's one shared `Base` subobject. Adds a vbase pointer / offset — more overhead.
- Interface segregation: many small abstract bases (mixins) over one fat one.

Prefer composition over inheritance — inheritance is the tightest coupling in the language. Reach for it only for genuine substitutable is-a relationships or to implement runtime polymorphism through a stable interface.

Inheritance is substitutability, not reuse. Public inheritance is an is-a contract (Liskov: anywhere an `A` is expected, a `B` must work). When you only want to *reuse* `A`'s methods, compose — make `A` a member of `B` (a *have-a*). The classic anti-pattern is `stack` deriving from `vector` to reuse `push_back`; now callers can `insert` into the middle of your "stack" and break its invariant. If the derived type can't honor every promise of the base, the relationship is have-a, and composition is the tool.

6. RUNTIME TYPE & CASTS

- `dynamic_cast` — safe down/cross-cast in a polymorphic hierarchy; returns null (pointers) or throws `bad_cast` (references) on failure. Uses RTTI, has a cost; frequent `dynamic_cast` often signals a missing virtual function.
 - `static_cast` — compile-time, no check (you assert the relationship).
 - `reinterpret_cast` — bit reinterpretation; almost always UB to then use as the new type (strict aliasing).
 - `const_cast` — add/remove `const`; modifying a truly-const object is UB.
 - `typeid` / `std::type_info` — runtime type identity (also RTTI).
-

7. STATIC VS DYNAMIC POLYMORPHISM (DESIGN TRADEOFF)

	Dynamic (virtual)	Static (templates/CRTP)
Dispatch	runtime, via vtable	compile time
Cost	indirection, no inline	zero-overhead, inlinable
Flexibility	heterogeneous containers, plugins, ABI boundary	one type per instantiation, code bloat
Errors	link/runtime	compile time

Say: "Virtual when the set of types varies at runtime or crosses an ABI boundary; CRTP/templates when types are known at compile time and I want zero overhead."

C++23 deducing `this` makes the static path far less painful — the base can recover the derived type without the CRTP template-parameter dance:

```
struct Base {  
    void run(this auto&& self) { self.impl(); }    // 'self' is the real  
    derived type  
};  
struct D : Base { void impl() { /*...*/ } };  
D{}.run();    // calls D::impl — static dispatch, zero overhead, no  
CRTP<Derived>
```

Take-home lesson. The virtual/CRTP choice is really a "when do I know the type?" question. Runtime → virtual (pay the vptr, gain flexibility). Compile time → templates/CRTP/deducing-`this` (pay in code size, gain inlining and zero dispatch cost).

SELF-CHECK LADDER (GRADE YOUR OWN MASTERY)

Level 1 — Recall

- What is a vptr? A vtable? Which is per-object and which is per-class?
- State the virtual-destructor rule in one sentence.
- What does `override` check that a bare matching signature doesn't?

Level 2 — Apply

1. Draw the object layout + vtable for a two-level hierarchy; trace a virtual call step by step.
2. Show the UB from a non-virtual base destructor; fix it two ways.
3. Demonstrate slicing and the by-reference fix.

Level 3 — Transfer

4. Why does a virtual call in a constructor not dispatch to the derived override? Explain via when the vptr is set.
5. Resolve a diamond with virtual inheritance; explain the added cost in the object layout.
6. Replace a `dynamic_cast`-heavy design with a virtual function — and say what the frequent `dynamic_cast` was really signaling.

Level 4 — Teach

- Explain to a Python dev why C++ objects don't "just know their own type" for free, and what the vtable buys and costs. Then explain when you'd choose CRTP instead.

CONNECTS TO

- File 01 — the virtual-destructor rule is a lifetime rule; `shared_ptr` vs `unique_ptr` differ in how they route destruction.
- File 03 — CRTP / static polymorphism is the compile-time counterpart to virtual dispatch.
- File 02 — slicing is a copy problem; passing by value copies the base subobject only.
- File 08 — `variant` + `visit` is the closed-set alternative to an open virtual hierarchy.